Labo 10 — Labo pratique : exploiter la stack

Objectif : sur la stack du labo 09, lire les logs et les métriques, poser des limites et provoquer un OOM, graver un `HEALTHCHECK` dans l'image (et découvrir pourquoi Podman le perd), durcir un conteneur, pousser dans un registry privé, nettoyer sans casser, puis écrire un Quadlet pour le démarrage au boot.

Prérequis — Labos 01 à 09 terminés : `~/labo-docker/09/` complet, images `api:1.0`, `postgres:16-alpine`, `nginx:alpine`, `alpine` ; podman-compose installé ; sur WSL, `systemd=true` (nécessaire pour `podman stats` et pour l'étape 8).

Fichiers fournis (`files/`) - `api.container` — le Quadlet de l'étape 8.

Étape 0 — Relancer la stack

```
mkdir -p ~/labo-docker/10 && cd ~/labo-docker/09 && podman compose up -d ; sleep 12 ; cd
~/labo-docker/10
podman compose -f ~/labo-docker/09/compose.yaml ps 2>/dev/null | grep -c healthy
```

Observez `2` : `db` et `api` sont `healthy`, `web` n'a pas de healthcheck.

Étape 1 — Lire les logs

```
podman logs -t --tail=2 stack_api_1
podman inspect stack_api_1 --format '{{.HostConfig.LogConfig.Type}}
{{.HostConfig.LogConfig.Path}}'
curl -s localhost:18080/api/ > /dev/null ; podman logs --tail=1 stack_web_1
```

Observez deux lignes horodatées (`2026-…T12:58:00.068793040Z Arguments recus : (aucun)`) ; le pilote `k8s-file` et un chemin `…/overlay-containers/<id>/userdata/ctr.log` ; puis la ligne d'accès nginx `"GET /api/ HTTP/1.1" 200 …` générée par votre `curl`.

Explication. Chaque conteneur a son fichier de log, écrit par `conmon`. `-t` ajoute l'horodatage, `--since` / `--tail` bornent la lecture. Ce fichier n'a pas de taille maximale par défaut.

Étape 2 — Processus, ressources, événements

```
podman top stack_api_1 user pid args
podman stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}'
podman inspect stack_api_1 --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}}
pids={{.HostConfig.PidsLimit}}'
podman events --stream=false --since 5m --filter container=stack_api_1 | cut -c1-60
```

Observez `podman 1 java -jar /app/api.jar` (l'utilisateur 1000 de l'image, vu avec votre nom) ; dans `stats`, quelques pour cent de CPU et `27.58MB / 32.48GB` : la « limite » est la mémoire de la machine ; `mem=0 cpus=0 pids=0` : aucune limite posée ; et la liste des événements `container init`, `container start`, puis un `health_status` toutes les cinq secondes.

Explication. `stats` lit les cgroups, `top` les processus, `inspect` la configuration, `events` le journal du moteur. Sans `--memory`, rien n'arrête ce conteneur avant la machine entière.

Étape 3 — Limiter, et provoquer un OOM

```
podman run -d --name api-limit --memory 256m --cpus 0.5 api:1.0 ; sleep 3
podman inspect api-limit --format 'mem={{.HostConfig.Memory}} cpus={{.HostConfig.NanoCpus}}'
podman stats --no-stream --format '{{.Name}} {{.MemUsage}} {{.MemPerc}}' api-limit
podman exec api-limit cat /sys/fs/cgroup/memory.max
podman rm -f -t 0 api-limit
```

Observez `mem=268435456 cpus=500000000` (octets et nano-CPU), `21.46MB / 268.4MB 8.00%`, et depuis l'intérieur du conteneur `memory.max` = `268435456` : la limite est un fichier cgroup que le processus peut lire.

```
podman run --name oom --memory 32m alpine sh -c 'head -c 100m /dev/zero | tail' ; echo
"exit=$?"
podman inspect oom --format 'oom={{.State.OOMKilled}} exit={{.State.ExitCode}}'
podman rm oom
```

Observez `Killed`, `exit=137`, puis `oom=true exit=137` : `tail` a voulu garder 100 Mo en mémoire dans 32 Mo, le noyau l'a tué.

```
podman run -d --name api-heap --memory 256m api:1.0 ; sleep 3
podman exec api-heap sh -c 'java -XX:+PrintFlagsFinal -version 2>/dev/null | grep "
MaxHeapSize"'
podman exec api-heap sh -c 'java -XX:MaxRAMPercentage=50 -XX:+PrintFlagsFinal -version
2>/dev/null | grep " MaxHeapSize"'
podman rm -f -t 0 api-heap
```

Observez `MaxHeapSize = 132120576` (126 Mo : sous une petite limite, la JVM prend par défaut la moitié, pas 25 %) puis `134217728` (exactement 128 Mo avec `MaxRAMPercentage=50`) : la JVM a lu la limite du conteneur et dimensionne son tas dessus.

Explication. `--memory` écrit `memory.max` dans le cgroup du conteneur ; le dépassement n'est pas une erreur applicative mais un `SIGKILL` du noyau. Pour Java, on règle le tas en pourcentage de cette limite, par `JAVA_TOOL_OPTIONS`.

Étape 4 — Un `HEALTHCHECK` dans l'image

```
printf 'FROM localhost/api:1.0\nHEALTHCHECK --interval=5s --timeout=3s --retries=3 CMD wget -
qO- http://localhost:8080/actuator/health || exit 1\n' > Dockerfile.health
podman build -q -f Dockerfile.health -t api:1.1 .
podman image inspect api:1.1 --format '{{.Config.Healthcheck}}'
podman build -q --format docker -f Dockerfile.health -t api:1.1 .
podman image inspect api:1.1 --format '{{.Config.Healthcheck.Test}}'
```

Observez au premier build un avertissement `HEALTHCHECK is not supported for OCI image format and will be ignored. Must use` docker `format`, puis `<nil>` — l'instruction a disparu — puis `[CMD-SHELL wget -qO- http://localhost:8080/actuator/health || exit 1]` après le build `--format docker`.

```
podman run -d --name api-h api:1.1 ; sleep 9
podman ps --filter name=api-h --format '{{.Names}} {{.Status}}'
podman healthcheck run api-h ; echo "exit=$?"
podman inspect api-h --format '{{.State.Health.Status}} {{len .State.Health.Log}}'
podman exec api-h sh -c 'kill 1' ; sleep 1 ; podman ps -a --filter name=api-h --format
'{{.Status}}'
podman rm -f -t 0 api-h
```

Observez `api-h Up 9 seconds (healthy)`, `exit=0`, `healthy` avec déjà deux vérifications dans l'historique ; et après `kill 1`, `Exited (143) 1 second ago (healthy)` : le healthcheck n'a rien relancé, c'est un indicateur — qui garde même son dernier état.

Explication. Le format OCI n'a pas de champ pour le healthcheck ; Buildah le laisse tomber avec un avertissement d'une ligne. Avec `--format docker`, l'image porte sa propre sonde, exécutée par un timer systemd en rootless.

Étape 5 — Durcir

```
podman run --rm --entrypoint id api:1.0
podman run --rm --entrypoint sh api:1.0 -c 'touch /app/x && echo "ecriture OK dans /app"'
podman run --rm --read-only --tmpfs /tmp --entrypoint sh api:1.0 -c 'touch /app/x || echo
"lecture seule" ; touch /tmp/x && echo "tmp OK"'
```

Observez `uid=1000(…) gid=1000(…)` : l'image tourne déjà en non-root (`USER 1000:1000`, labo 05) ; `touch: cannot touch '/app/x': Permission denied` (le dossier appartient à root) ; puis avec `--read-only`, `Read-only file system` partout sauf dans le `tmpfs`.

```
podman run --rm --entrypoint sh alpine -c 'chown nobody /etc/hostname && echo "chown OK (root
avec capabilities)"'
podman run --rm --cap-drop ALL --entrypoint sh alpine -c 'chown nobody /etc/hostname'
podman run --rm --security-opt no-new-privileges --cap-drop ALL --read-only --tmpfs /tmp --user
1000 --entrypoint id alpine
```

Observez `chown OK` pour le root ordinaire du conteneur, `chown: /etc/hostname: Operation not permitted` sans capabilities, et `uid=1000 gid=0(root)` pour la ligne complète de durcissement — qui démarre sans erreur.

Explication. Non-root, lecture seule, zéro capability, pas d'escalade : quatre options, aucune ligne de code. Ces options ont leur équivalent exact dans Compose et dans Kubernetes.

Étape 6 — Un registry privé

```
podman run -d --name registry -p 5000:5000 docker.io/library/registry:2 ; sleep 2
podman tag api:1.0 localhost:5000/api:1.0
podman push localhost:5000/api:1.0 2>&1 | tail -1 | cut -c1-120
podman push --tls-verify=false localhost:5000/api:1.0 2>&1 | tail -1
curl -s localhost:5000/v2/_catalog ; echo ; curl -s localhost:5000/v2/api/tags/list ; echo
```

Observez `Error: … Get "https://localhost:5000/v2/": http: server gave HTTP response to HTTPS client`, puis `Writing manifest to image destination`, et l'API du registry : `{"repositories":["api"]}` et `{"name":"api","tags":["1.0"]}`.

```
podman rmi localhost:5000/api:1.0
podman pull --tls-verify=false localhost:5000/api:1.0 2>&1 | tail -1
podman images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep 'api:1.0'
podman image inspect api:1.0 --format '{{.Digest}}'
podman rm -f -t 0 -v registry
```

Observez après l'aller-retour, `localhost:5000/api:1.0` et `localhost/api:1.0` avec le **même** `IMAGE ID` ; un digest `sha256:52b82ed9…` (le vôtre diffère) ; et `rm -v`, car `registry:2` déclare un `VOLUME` (labo 06).

Explication. Le registry stocke des blobs adressés par leur hachage ; ce qui en revient est bit pour bit ce qui y est entré. TLS est exigé par défaut ; `--tls-verify=false` n'est acceptable que pour ce test.

Étape 7 — Mesurer, puis nettoyer

```
podman system df
podman images --filter dangling=true --format '{{.ID}} {{.Size}}'
podman image prune -f
podman system df | head -2
```

Observez le bilan par catégorie avec la colonne `RECLAIMABLE` ; la liste des images `<none>` (chez vous, probablement les restes des rebuilds de `api:1.1`) ; leurs ID supprimés ; et un total d'images qui baisse. Les volumes et les conteneurs n'ont pas été touchés.

Explication. `image prune` sans `-a` ne retire que les images dangling : c'est le seul `prune` à lancer par réflexe. Les autres (`-a`, `container`, `volume`, `system`) se décident après lecture de `system df` et de `volume ls --filter dangling=true`, et la suppression d'un volume se fait **par nom**.

Étape 8 — Démarrer au boot avec Quadlet (WSL)

Cette étape exige systemd dans votre distribution (`systemd=true` dans `/etc/wsl.conf`) ; elle ne peut pas être rejouée dans le bac à sable de rédaction, les sorties ci-dessous sont donc décrites et non citées.

```
mkdir -p ~/.config/containers/systemd
cp <chemin-du-labo>/files/api.container ~/.config/containers/systemd/
systemctl --user daemon-reload
systemctl --user start api
systemctl --user status api --no-pager | head -5
podman ps --format '{{.Names}} {{.Status}} {{.Ports}}'
curl -s localhost:18080/actuator/health ; echo
journalctl --user -u api -n 3 --no-pager
```

Observez un service `api.service` `active (running)`, un conteneur nommé `systemd-api` (préfixe ajouté par Quadlet) publié sur `18080`, `{"status":"UP"}`, et les logs de l'API dans le journal systemd.

```
systemctl --user enable api
loginctl enable-linger $USER
systemctl --user stop api
podman ps -a --format '{{.Names}}' | grep -c systemd-api
```

Observez `0` après `stop` : Quadlet lance le conteneur avec `--rm`, l'arrêt du service le supprime ; au prochain démarrage de la session (ou de WSL, une fois la distribution lancée), systemd le recrée.

Explication. Le fichier `.container` est traduit en unité systemd ; c'est systemd, relancé par le système, qui relance Podman. `enable-linger` maintient la session `--user` sans connexion ouverte. Pour la suite des labos, laissez le service **arrêté** (`systemctl --user disable --now api`) afin que le port 18080 reste libre.

Nettoyage

```
cd ~/labo-docker/09 && podman compose down && cd ~/labo-docker/10
podman rmi api:1.1 localhost:5000/api:1.0 docker.io/library/registry:2
rm -f Dockerfile.health
podman ps -a --format '{{.Names}}' ; podman volume ls --format '{{.Name}}' ; podman images --
format '{{.Repository}}:{{.Tag}}'
```

Observez aucun conteneur, le volume `stack_pgdata` conservé (la base du labo 09), et les images de la stack toujours là. Fin de la partie Docker : la partie Kubernetes (labo 11) réutilise `api:1.0`, `nginx:alpine` et `postgres:16-alpine`.

Ce que vous devez pouvoir affirmer maintenant

- `logs -t`, `top`, `stats`, `events` : le fichier `ctr.log`, le processus `java` en UID 1000, `27 MB` utilisés sans limite, un `health_status` toutes les cinq secondes.
- `--memory 256m` = `memory.max` lisible depuis le conteneur ; `32m` pour 100 Mo = `Killed`, `137`, `OOMKilled: true` ; la JVM dimensionne son tas sur la limite, `MaxRAMPercentage` fixe la part.
- `HEALTHCHECK` perdu au format OCI (`<nil>`), conservé avec `--format docker` (`(healthy)`), et il ne relance rien (`Exited (143)`).

- `--read-only` + `--tmpfs`, `--cap-drop ALL` (`Operation not permitted`), `no-new-privileges`, non-root : la ligne durcie démarre.
- `push` refusé sans TLS, accepté avec `--tls-verify=false` ; même `IMAGE ID` après l'aller-retour ; `rm -v` pour le registry.
- `system df` d'abord, `image prune` (dangling) par réflexe, le reste par nom.
- Quadlet : `.container` → `api.service`, `enable` + `enable-linger` pour survivre à `wsl --shutdown`.